BORNFIRE

PREVIOUS MODULES & ARCHITECTURE GUIDE

TECHNICAL REFERENCE

AUTHOR: BORNFIRE ENGINEERING

TARGET AUDIENCE: INTERNS / NEW DEVELOPERS

1. STATE MANAGEMENT ARCHITECTURE

The client-side state is orchestrated primarily using React Contexts located in `frontend/src/contexts/`. This approach guarantees that user authenticated state, tasks, habit configurations, and live connections are shared across distinct visual pages.

1.1 AUTHENTICATION (AUTHCONTEXT)

Managed by the `AuthProvider` in `AuthContext.tsx`. It exposes:

- `user` and `session` objects fetched from `@supabase/supabase-js`.
- Signup logic (`signUpWithEmail`) which triggers profile database updates under the `users` table, generating unique IDs using `#BF-`.
- Login options (Google/GitHub oauth redirect states).

1.2 CORE ACTIONS & TASK STATE (TASKCONTEXT)

Managed by the `TaskProvider` in `TaskContext.tsx`. This acts as the state heart, supplying:

- Active tasks, UserProfile statistics, squad/friend arrays, and leaderboard listings.
- Timer actions (`startTimer`, `stopTimer`, `updateTaskTime`) to run the Pomodoro focus loop.
- State synchronization wrappers that keep both LocalStorage key values and Supabase database tables in sync.

2. DUAL-LAYER STORAGE SYNCHRONIZATION

To prevent work loss on reload and enable real-time tracking, Bornfire runs a dual-layer synchronization pattern:

> ⚠️ **DUAL-LAYER STORAGE PATTERN**
>
> Local storage is treated as the source of truth for UI actions to eliminate input lag. Background hooks synchronize these states to the cloud database (Supabase) in a debounced, gentle sequence.

2.1 UPSTREAM SYNC (CLIENT TO DATABASE)

Whenever tasks change locally (timers increment, completion toggles), a debounced useEffect (3-second delay) runs in the background. It finds database tasks that have been deleted locally, deletes them, and upserts changed tasks to the cloud:

```
// Debounced Task Sync Hook
useEffect(() => {
  if (!user) return;
  const syncTasks = async () => {
    // 1. Sync deletions
    // 2. Upsert changed elements
  };
  const handler = setTimeout(syncTasks, 3000);
  return () => clearTimeout(handler);
}, [tasks, user]);
```

2.2 DOWNSTREAM SYNC (DATABASE TO CLIENT)

During application initialization, if the user logs in and their local task store is empty, database tasks are queried and populate the state. Real-time updates for the squad and leaderboard are refreshed every 15 seconds.

3. CORE VIEW IMPLEMENTATIONS

3.1 SIDEBAR & PROFILE STATUS CARD

Located in `frontend/src/components/layout/Sidebar.tsx`. It lists the main navigation links, the PWA download button, and displays a user profile card containing the current user level and league badge.

3.2 WATCH FRIENDS (SQUAD WATCH PAGE)

Located in `frontend/src/pages/Friends.tsx`. Users search for squad buddies using their unique Bornfire ID. The squad is split visually:

- **Live Status**: Squad buddies whose `updated_at` timestamp was updated within 90 seconds. Shows if they are currently working (running focus timer) and progress percentage.
- **Stationary**: Buddies who are currently offline.

3.3 LEADERBOARDS & LEAGUES

Located in `frontend/src/pages/Leaderboard.tsx`. Ranks users by cumulative XP. Users fall into gamified league tiers:

- Bronze (Level 1 - 19)
- Silver (Level 20 - 39)
- Gold (Level 40 - 59)
- Platinum (Level 60 - 79)
- Diamond (Level 80+)